

tiling

May 9, 2024

1 Motivation of the study

The question is motivated by a educational example of preparing a 2D matrix for tiled MatMul. In particular, suppose we have an input matrix A of size 6x6 and we would like to apply a 2x2 tiling on A to get a matrix B of size (3,3,2,2). Then re-arrange B in memory to have tiles in the contiguous memory, we name the tiled matrix C . We demonstrate two different ways, which rely on different operations in numpy, that will achieve the same results. We raise a question is there some principled way to prove such two ways are equivalent ?

2 First way of pre and post processing matrix for tiled MatMul()

```
[3]: import numpy as np
A = np.arange(36, dtype=np.float32).reshape(6,6)
```

```
[4]: print(A)

[[ 0.  1.  2.  3.  4.  5.]
 [ 6.  7.  8.  9. 10. 11.]
 [12. 13. 14. 15. 16. 17.]
 [18. 19. 20. 21. 22. 23.]
 [24. 25. 26. 27. 28. 29.]
 [30. 31. 32. 33. 34. 35.]]
```

```
[5]: B=np.lib.stride_tricks.as_strided(A, shape=(3,3,2,2), strides=np.
    ↪array((12,2,6,1))*4) # tile A to get B
C=np.ascontiguousarray(B)
```

```
[6]: print(C.reshape(6,6)) # if we look at the first 4 elements 0,1,6,7 ,
    # they are indeed the first tile of A, so on and so forth
```

```
[[ 0.  1.  6.  7.  2.  3.]
 [ 8.  9.  4.  5. 10. 11.]
 [12. 13. 18. 19. 14. 15.]
 [20. 21. 16. 17. 22. 23.]
 [24. 25. 30. 31. 26. 27.]
 [32. 33. 28. 29. 34. 35.]]
```

```
[7]: # Now call into C function to do the tiled MatMul that involves C...
      # After tiled MatMul is done, we need to convert C back into the original
      ↪ memory layout
      D=np.transpose(C, axes=(0,2,1,3)) # We can invert C back to A by permuting the
      ↪ 1st and 2nd axis
      print(D.reshape(6,6))

[[ 0.  1.  2.  3.  4.  5.]
 [ 6.  7.  8.  9. 10. 11.]
 [12. 13. 14. 15. 16. 17.]
 [18. 19. 20. 21. 22. 23.]
 [24. 25. 26. 27. 28. 29.]
 [30. 31. 32. 33. 34. 35.]]
```

3 Second way of pre and post processing matrix for tiled MatMul()

```
[8]: A = np.arange(36, dtype=np.float32).reshape(6,6)

[9]: B = A.reshape(3,2,3,2)
      C = np.transpose(B, axes=(0,2,1,3)) # we can replace the as_strided() call with
      ↪ a reshape and transpose
      print(C.reshape(6,6))

[[ 0.  1.  6.  7.  2.  3.]
 [ 8.  9.  4.  5. 10. 11.]
 [12. 13. 18. 19. 14. 15.]
 [20. 21. 16. 17. 22. 23.]
 [24. 25. 30. 31. 26. 27.]
 [32. 33. 28. 29. 34. 35.]]
```

Now we can see C matrix in First way and C matrix in Second way are exactly the same, before being sent to the tiled MatMul routine. Here is the question: is there some matrix calculus that we can rely on to tell the equivalence of certain operations (e.g., `as_strided` vs `reshape+transpose`) given the proper shape. Or, we have to rely on knowing the time-tested tricks ?

I am not sure if we can define things like `reshape()`, `transpose()` as linear operators and then find their inverse operators – but I tend to think they are nothing but identity operators if we linearize each tensor in the vector space. Further, operations such as `as_strided()` might not even be invertible.

```
[ ]:
```